LSTMCell#

https://pytorch.org/docs/stable/generated/torch.nn.LSTMCell.html

Description:

A long short-term memory (LSTM) cell. where σ\sigmaσ is the sigmoid function, and ⊙\odot⊙ is the Hadamard product. input_size (int) – The number of expected features in the input x hidden_size (int) – The number of features in the hidden state h bias (bool) – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

Function Signature

```
class torch.nn.LSTMCell(input_size, hidden_size, bias=True,
device=None, dtype=None)[source]#
Parameters
Inputs: input, (h_0, c_0)
Outputs: (h_1, c_1)
Variables
```

Code Examples

```
>>> rnn = nn.LSTMCell(10, 20)  # (input_size, hidden_size)
>>> input = torch.randn(2, 3, 10)  # (time_steps, batch,
input_size)
>>> hx = torch.randn(3, 20)  # (batch, hidden_size)
>>> cx = torch.randn(3, 20)
>>> output = []
>>> for i in range(input.size()[0]):
...     hx, cx = rnn(input[i], (hx, cx))
...     output.append(hx)
>>> output = torch.stack(output, dim=0)
```

Notes & Warnings

NOTE Note All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

Detailed Documentation

Documentation

A long short-term memory (LSTM) cell.

where σ\sigmaσ is the sigmoid function, and ⊙\odot⊙ is the Hadamard product.

input_size (int) – The number of expected features in the input x

hidden_size (int) – The number of features in the hidden state h

bias (bool) – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

input of shape (batch, input_size) or (input_size): tensor containing input features

h_0 of shape (batch, hidden_size) or (hidden_size): tensor containing the initial hidden state

c_0 of shape (batch, hidden_size) or (hidden_size): tensor containing the initial cell state

If (h_0, c_0) is not provided, both h_0 and c_0 default to zero.

h_1 of shape (batch, hidden_size) or (hidden_size): tensor containing the next hidden state

c_1 of shape (batch, hidden_size) or (hidden_size): tensor containing the next cell state

weight_ih (torch.Tensor) – the learnable input-hidden weights, of shape (4*hidden_size, input_size)

weight_hh (torch.Tensor) – the learnable hidden-hidden weights, of shape (4*hidden_size, hidden_size)

bias_ih – the learnable input-hidden bias, of shape (4*hidden_size)

bias_hh – the learnable hidden-hidden bias, of shape (4*hidden_size)

All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k ,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.
